



deeplearning.ai

# Optimization Algorithms

---

Mini-batch  
gradient descent

# Batch vs. mini-batch gradient descent

Vectorization allows you to efficiently compute on  $m$  examples.

# Mini-batch gradient descent



deeplearning.ai

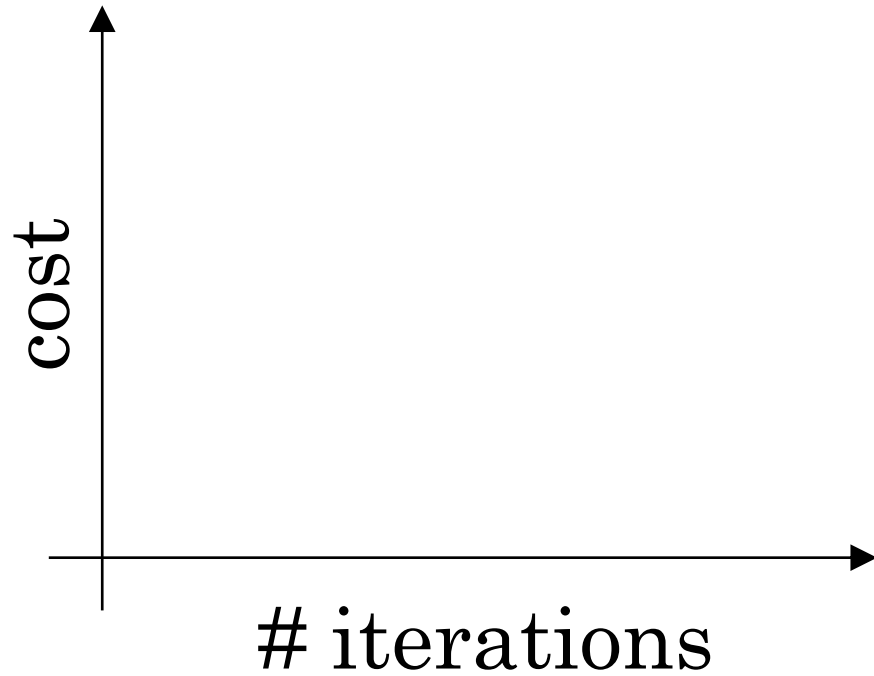
# Optimization Algorithms

---

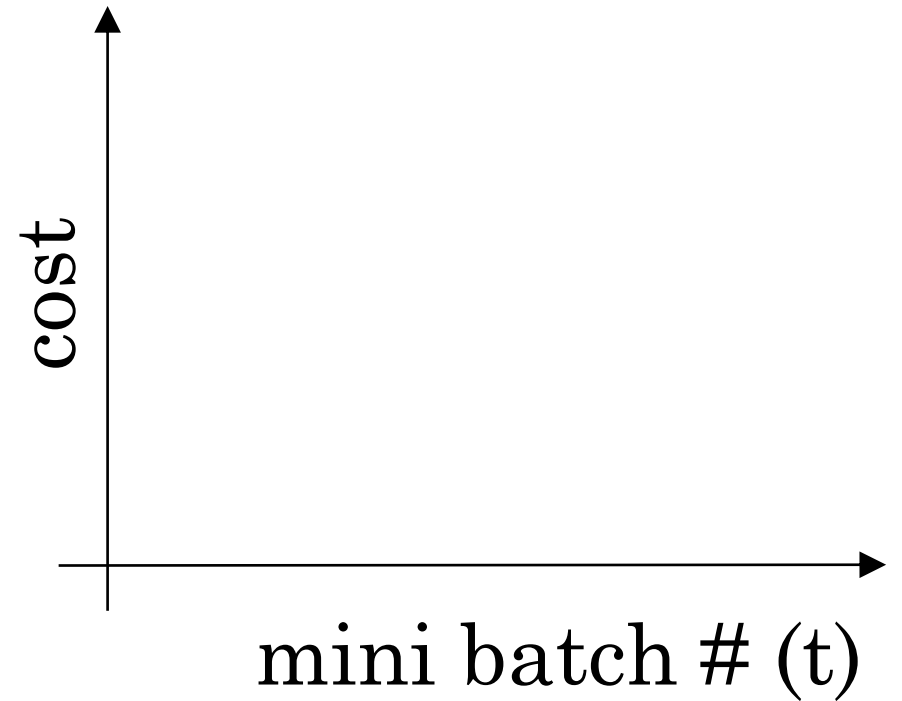
Understanding  
mini-batch  
gradient descent

# Training with mini batch gradient descent

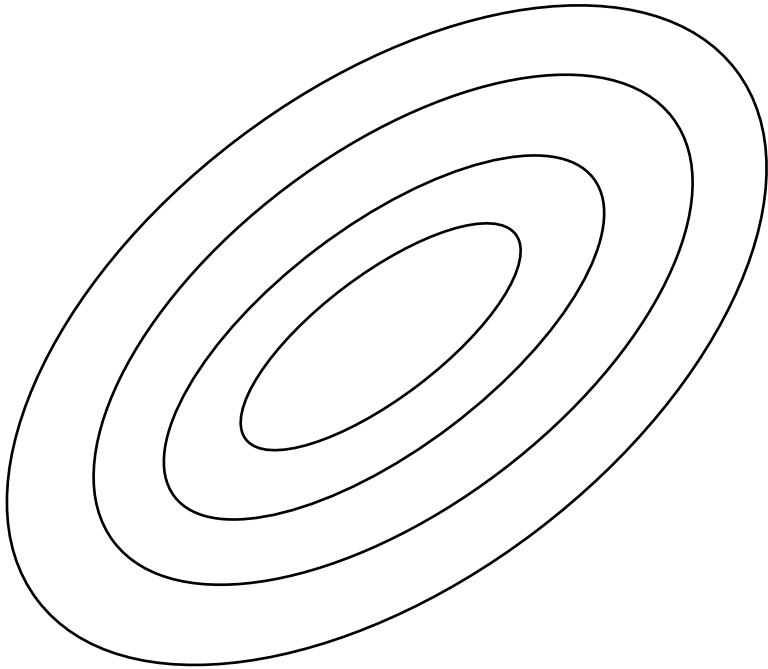
Batch gradient descent



Mini-batch gradient descent



# Choosing your mini-batch size



# Choosing your mini-batch size



deeplearning.ai

# Optimization Algorithms

---

## Exponentially weighted averages



# Temperature in London

$$\theta_1 = 40^\circ\text{F}$$

$$\theta_2 = 49^\circ\text{F}$$

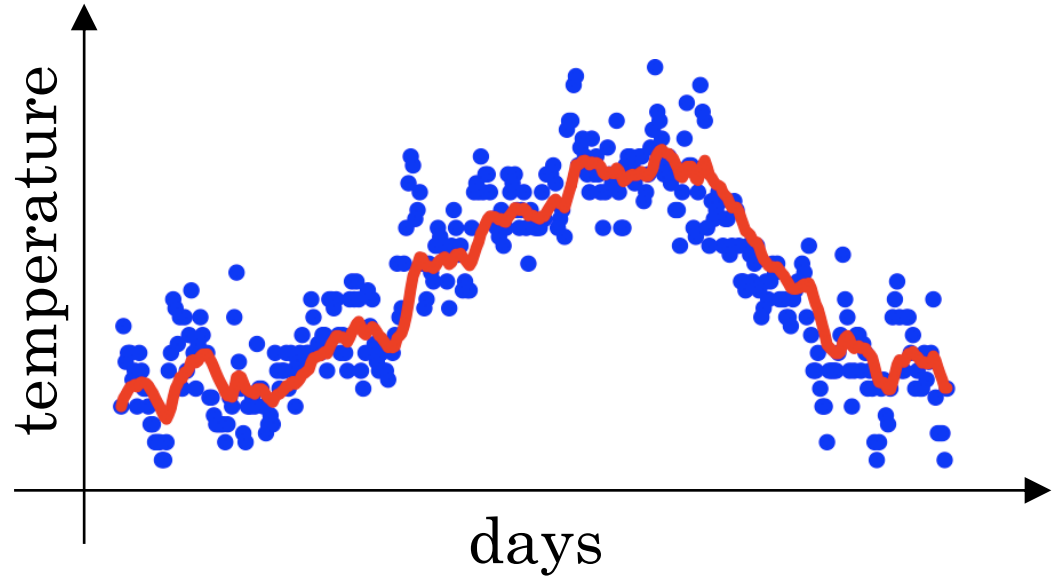
$$\theta_3 = 45^\circ\text{F}$$

$\vdots$

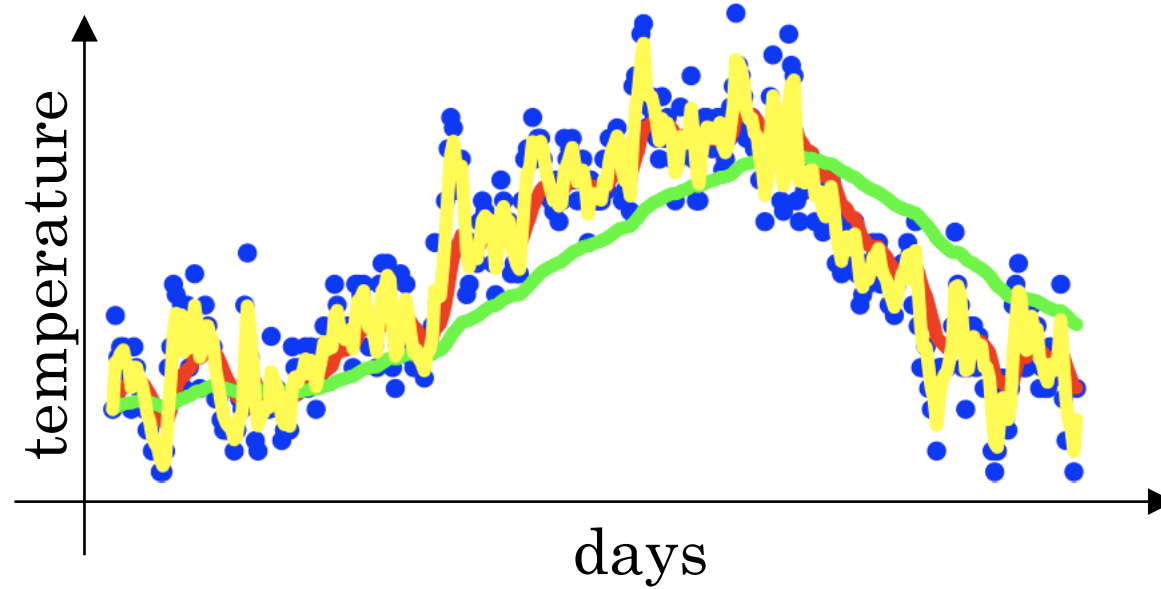
$$\theta_{180} = 60^\circ\text{F}$$

$$\theta_{181} = 56^\circ\text{F}$$

$\vdots$



# Exponentially weighted averages





deeplearning.ai

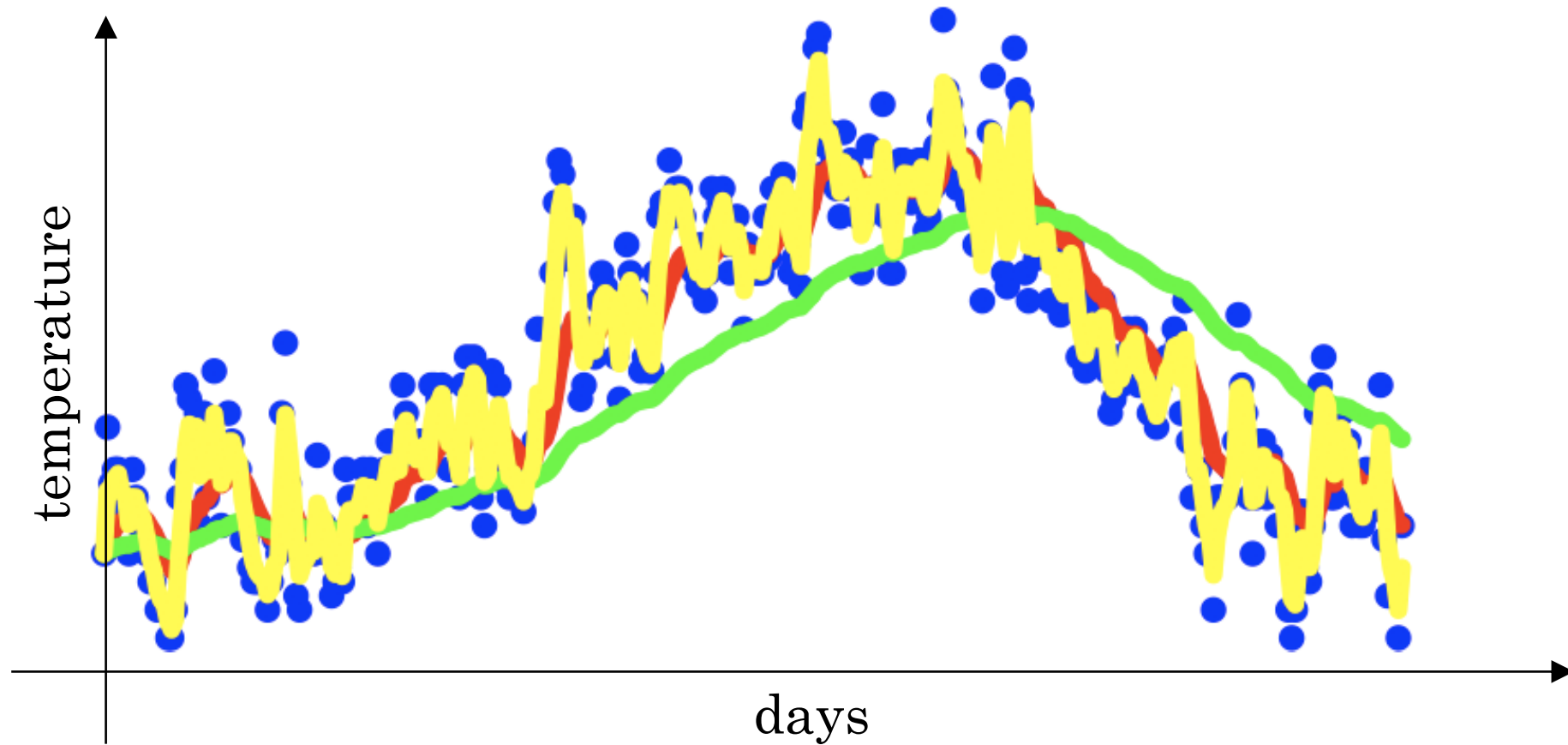
# Optimization Algorithms

---

Understanding  
exponentially  
weighted averages

# Exponentially weighted averages

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$



# Exponentially weighted averages

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$

$$v_{100} = 0.9v_{99} + 0.1\theta_{100}$$

$$v_{99} = 0.9v_{98} + 0.1\theta_{99}$$

$$v_{98} = 0.9v_{97} + 0.1\theta_{98}$$

...

# Implementing exponentially weighted averages

$$v_0 = 0$$

$$v_1 = \beta v_0 + (1 - \beta) \theta_1$$

$$v_2 = \beta v_1 + (1 - \beta) \theta_2$$

$$v_3 = \beta v_2 + (1 - \beta) \theta_3$$

...



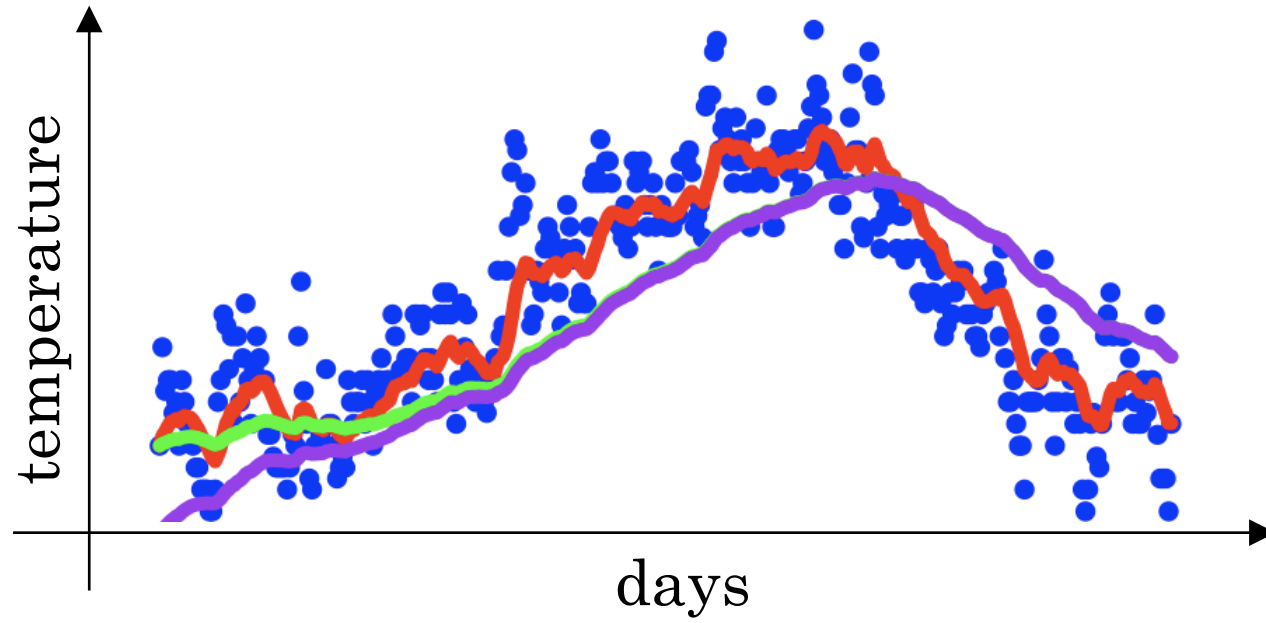
deeplearning.ai

# Optimization Algorithms

---

Bias correction  
in exponentially  
weighted average

# Bias correction



$$v_t = \beta v_{t-1} + (1 - \beta) \theta_t$$





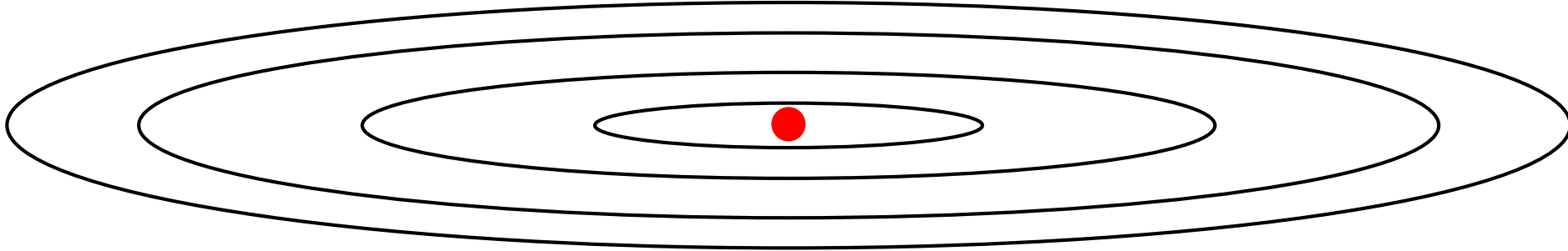
deeplearning.ai

# Optimization Algorithms

---

## Gradient descent with momentum

# Gradient descent example



deeplearning.ai

# Implementation details

On iteration  $t$ :

Compute  $dW, db$  on the current mini-batch

$$v_{dW} = \beta v_{dW} + (1 - \beta) dW$$

$$v_{db} = \beta v_{db} + (1 - \beta) db$$

$$W = W - \alpha v_{dW}, \quad b = b - \alpha v_{db}$$

Hyperparameters:  $\alpha, \beta$                        $\beta = 0.9$



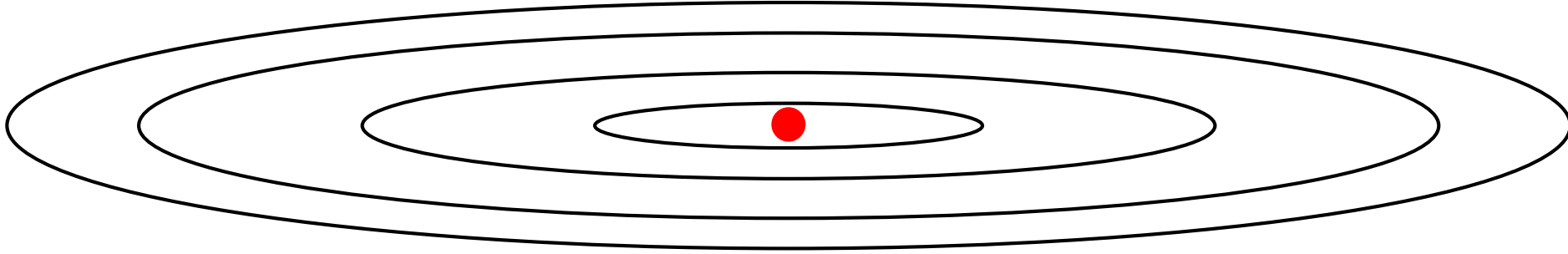
deeplearning.ai

# Optimization Algorithms

---

## RMSprop

RMSprop





deeplearning.ai

# Optimization Algorithms

---

## Adam optimization algorithm

# Adam optimization algorithm

```
yhat = np.array([.9, 0.2, 0.1, .4, .9])
```

# Hyperparameters choice:



Adam Coates





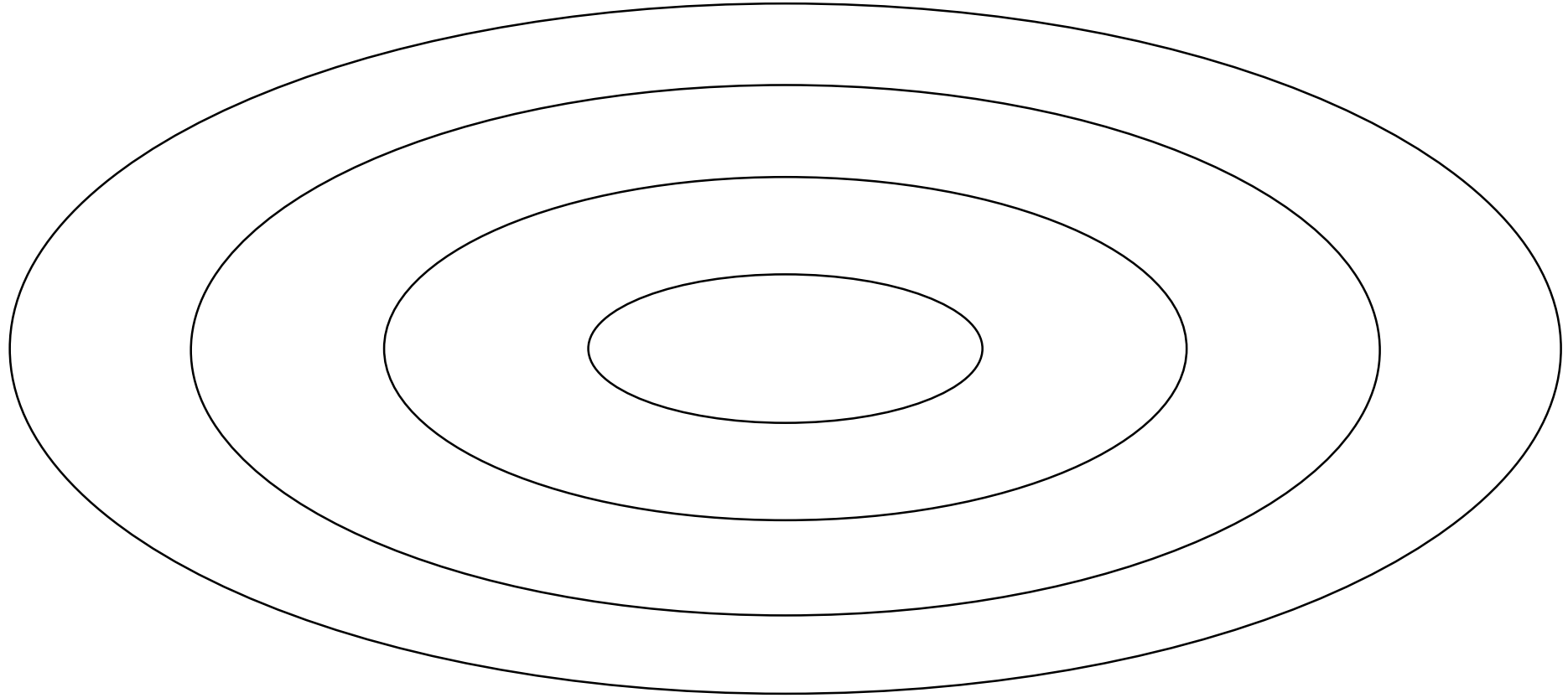
deeplearning.ai

# Optimization Algorithms

---

## Learning rate decay

# Learning rate decay



# Learning rate decay

# Other learning rate decay methods



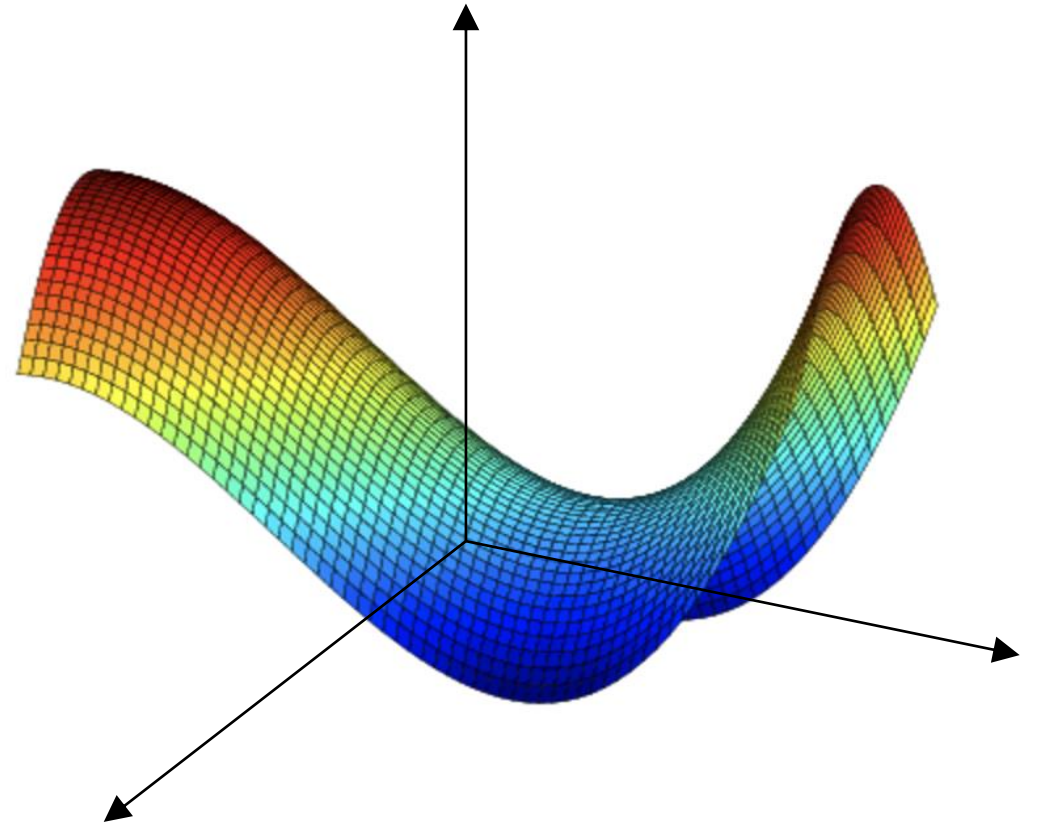
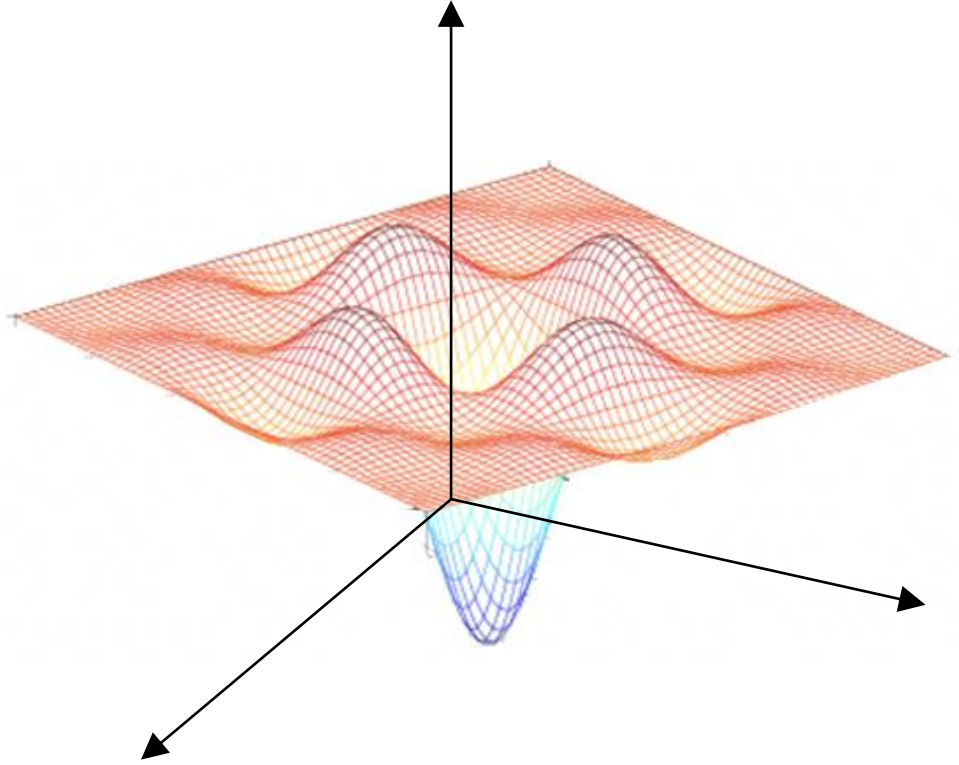
deeplearning.ai

# Optimization Algorithms

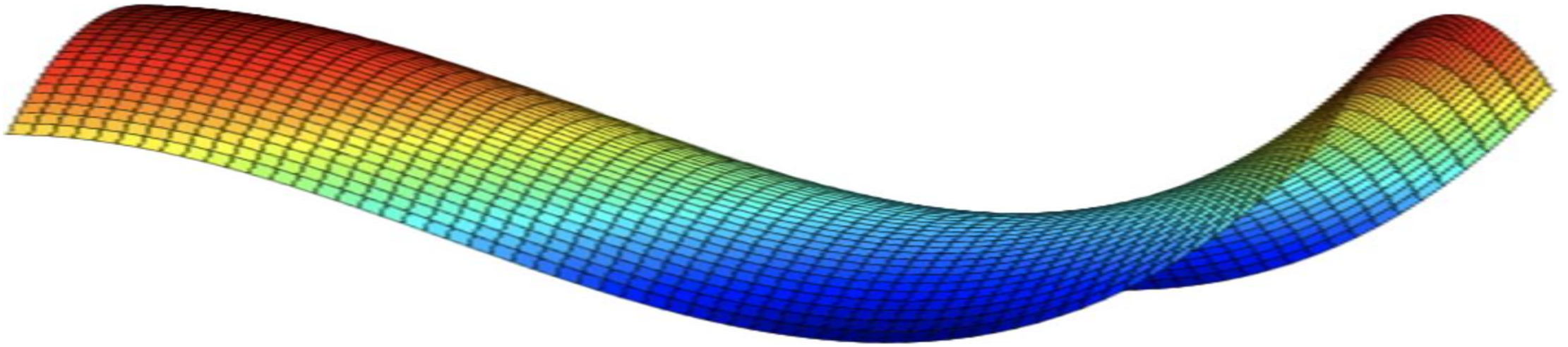
---

## The problem of local optima

# Local optima in neural networks



# Problem of plateaus



- Unlikely to get stuck in a bad local optima
- Plateaus can make learning slow